

Lab 7

[valid 2024-2025]

Concurrency

Write a program that simulates a *word game* between a given number of players.

At the beginning of the game there is a *bag* containing a number of *tiles*. Each tile represents a *letter* and has a number of *points*. The application will also use a *dictionary* that contains a list of acceptable *words*. Each player extracts 7 tiles from the bag and must create a word from the dictionary with them. When a player creates a word, it *submits* it to the *board* and receives a number of points. After submitting a word of length k , the player will immediately request other k tiles from the bag, if this is possible. If the player cannot create a word, it will discard all the tiles and extract others (and passes its turn). The game ends when the bag becomes empty. The winner is the player with the highest score.

The players might take turns (or not...) and a time limit might be imposed (or not...).

The main specifications of the application are:

Compulsory (1p)

- Create an object oriented model of the problem. You may assume that there are 10 tiles for each letter in the alphabet and each letter is worth a random number of points (between 1 and 10).
 - Each player will have a name and they must perform in a concurrent manner, extracting repeatedly tokens from the bag.
 - After each extraction, a player will submit to the board all its letters.
 - **Simulate the game using a thread for each player.**
Pay attention to the *synchronization* of the threads when extracting tokens from the bag and when putting words on the board.
-

Homework (2p)

- Create an implementation of a dictionary, using some collection of words. Use an appropriate data structure to represent the dictionary.
You may use [aspell](#) to generate a text file containing English words, or anything similar: [WordNet](#), [dexonline](#), etc.
 - Implement the logic of the game and determine who the winner is at the end.
 - Make sure that players wait their turns, using a *wait-notify* approach.
 - Implement a *timekeeper* thread that runs concurrently with the player threads, as a *daemon*. This thread will display the running time of the game and it will stop the game if it exceeds a certain time limit.
-

Bonus (2p)

- Using [Graph4J](#) create a labeled directed graph representing the words in the dictionary as a [prefix tree](#).
- Create a [DFS iterator](#) in order to display all the nodes of the tree.
- The dictionary must offer the possibility to search not only for a word, but for words which start with a given prefix (lookup). Implement the prefix search:
 - using a multi-threaded approach on the collection storing the words (parallel streams, ForkJoin, etc) and
 - using the prefix tree.

After creating the prefix tree, compare the performance of the two lookup implementations using a large dictionary (you may generate a random one with at least 1_000_000 words).

References

- [Slides](#)
- [The Java Tutorials: Concurrency](#)
- [Java Language Specification: Threads and Locks](#)
- [Java Concurrency / Multithreading Tutorial](#)

Objectives

- Understand the basic principles of *concurrent programming*.
- Create and run threads using the *Thread* class and the *Runnable* interface.
- Understand *resource contention* and *thread interference*.
- Create *synchronized* methods or statements.
- Implement the *wait-notify* mechanism.
- Understand the *thread pool* and *fork/join* concepts.